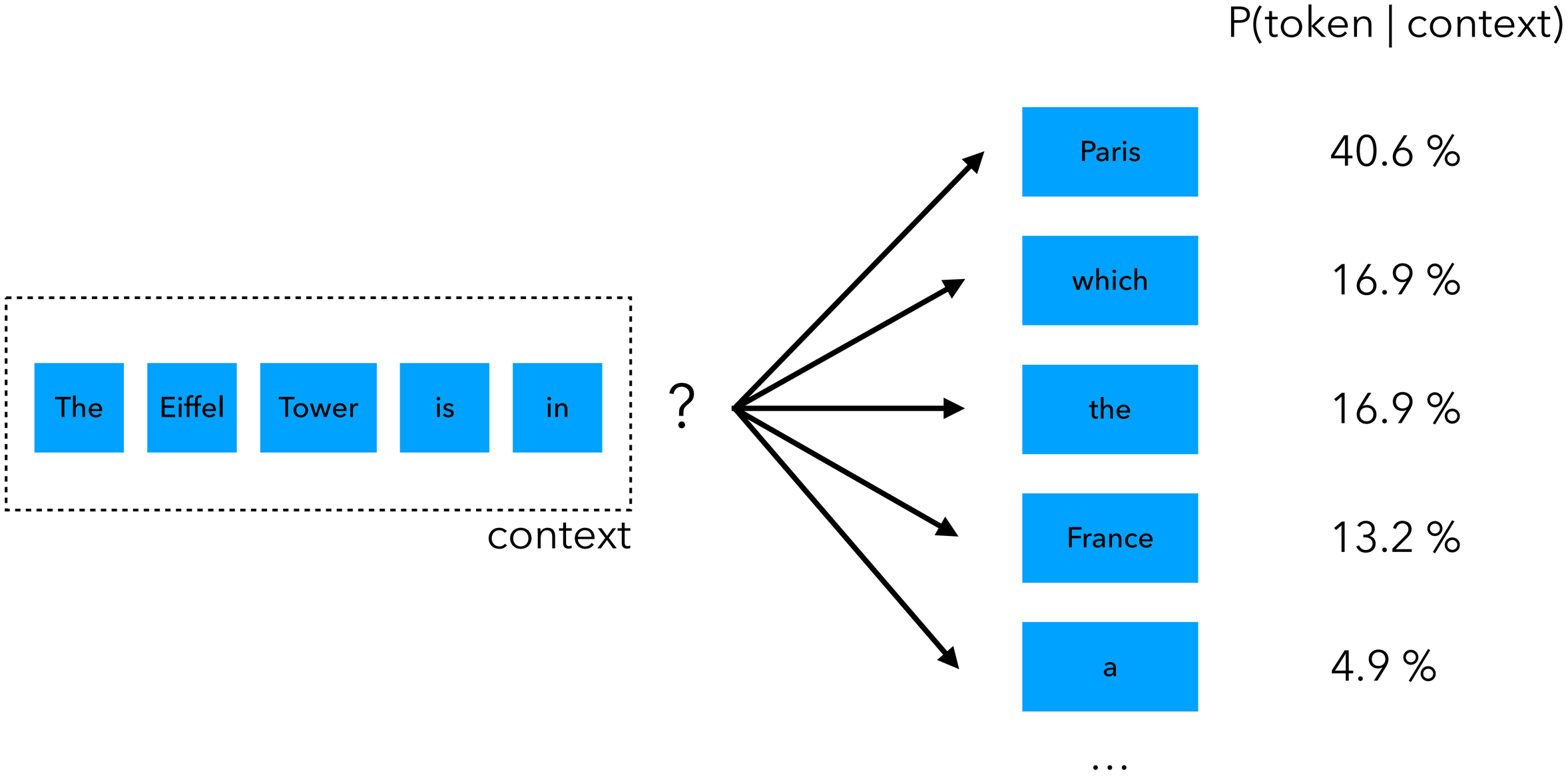
IA en perspective

Fondations et état des lieux

Fabien Rohrer - Freelance - MSc in Computer Science, EPFL

Génération de texte



Chaînes de Markov

Pour générer du texte

But: **prédit le prochain mot** à partir d'un **contexte** en utilisant des **probabilités**
Le prochain mot dépend uniquement des **n** derniers mots.

Fonctionnement

Corpus → calcul des probabilités

P(mot suivant | n derniers mots)

Génération :

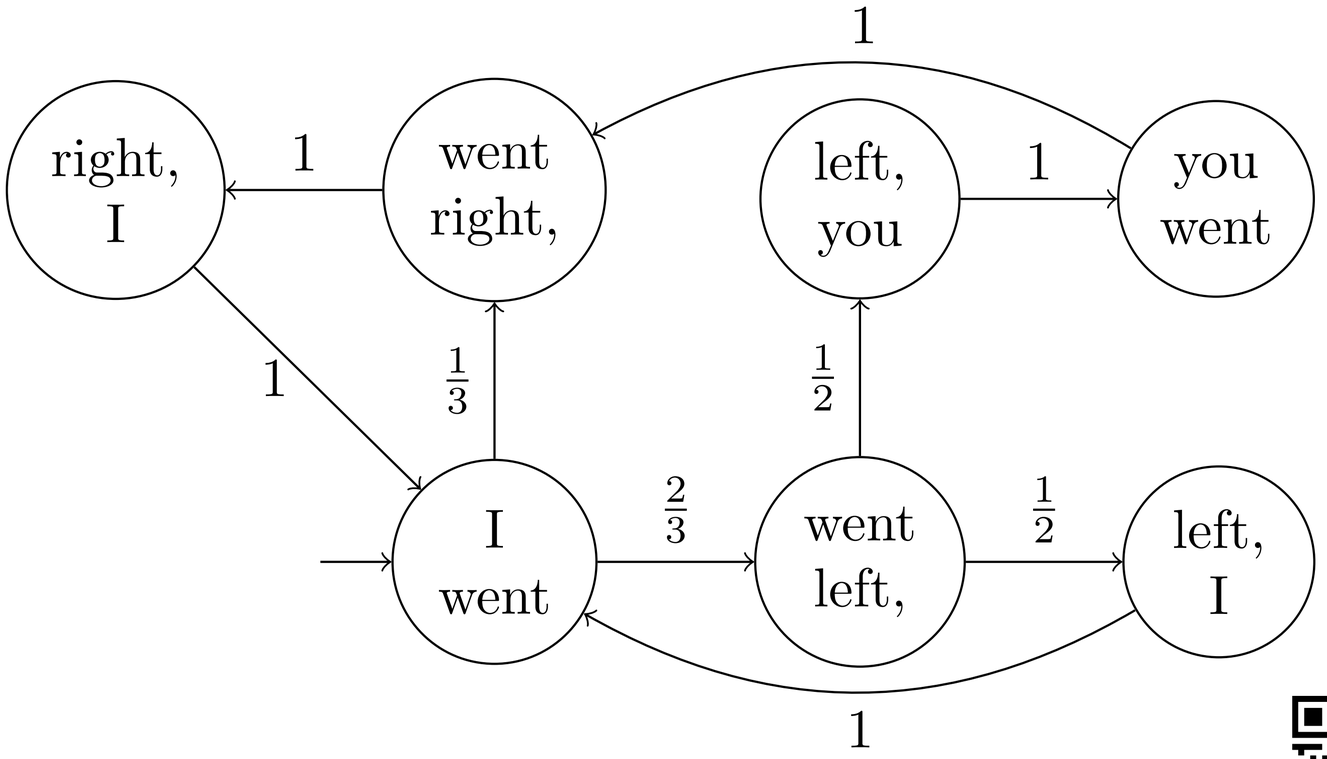
1. Choisir un état initial
2. Tirer le mot suivant selon les probabilités
3. Décaler
4. Répéter

État = n derniers mots

Ordre n = profondeur du contexte

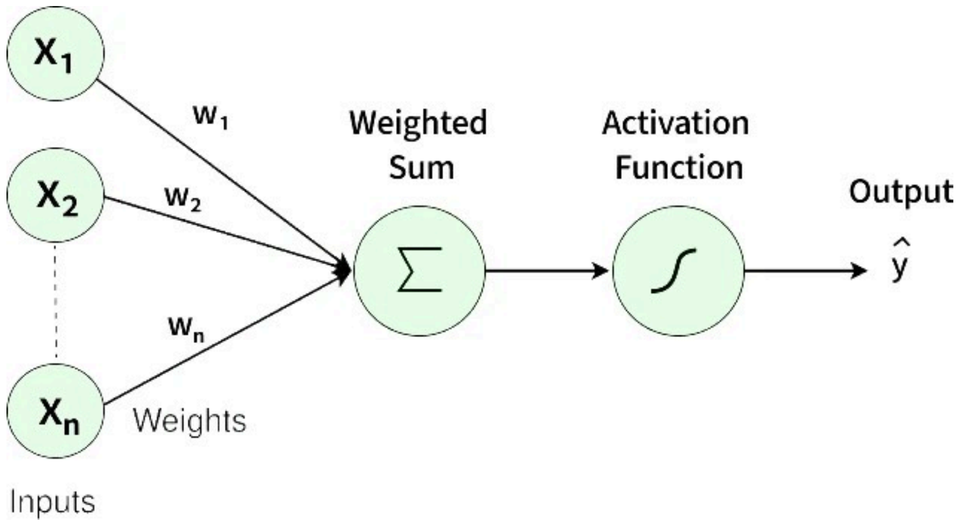
Modèle probabiliste, pas de compréhension

Marche aléatoire dans un graphe de transitions

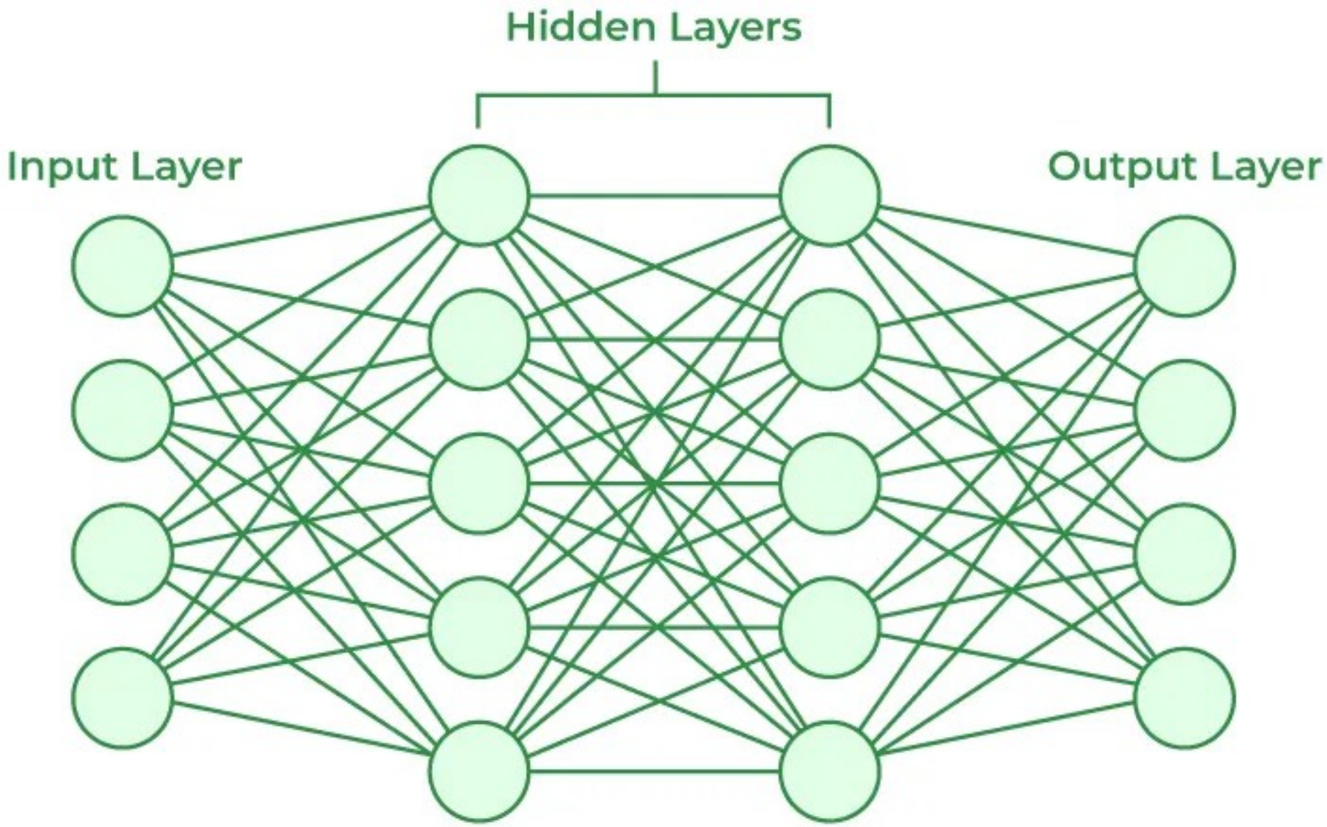


https://simulalgo.appspot.com/demo/markov/markov.html

Réseau de neurones artificiels



Artificial Neuron

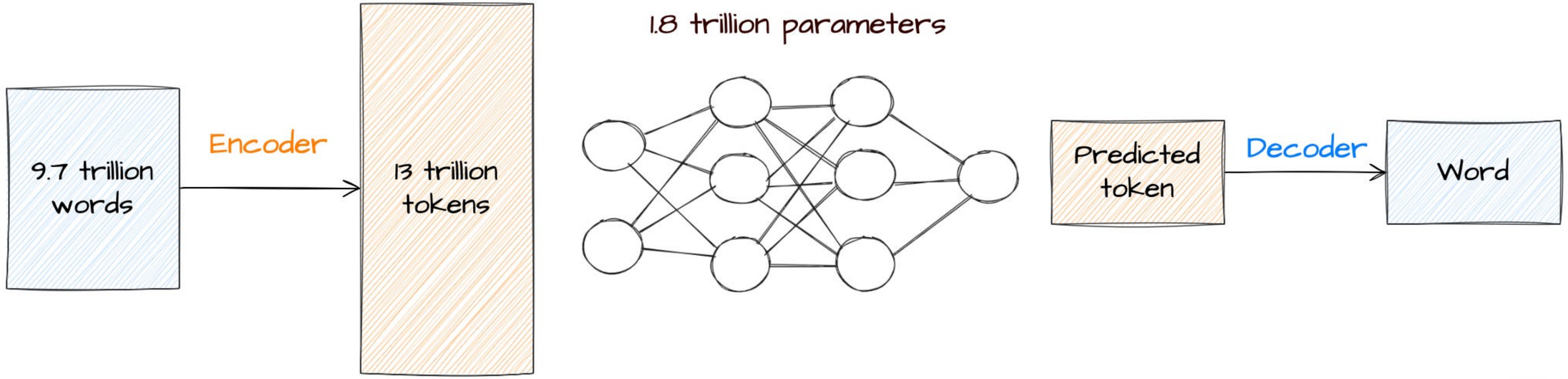


ANN
Backprop algorithm

LLM

Vue d'avion

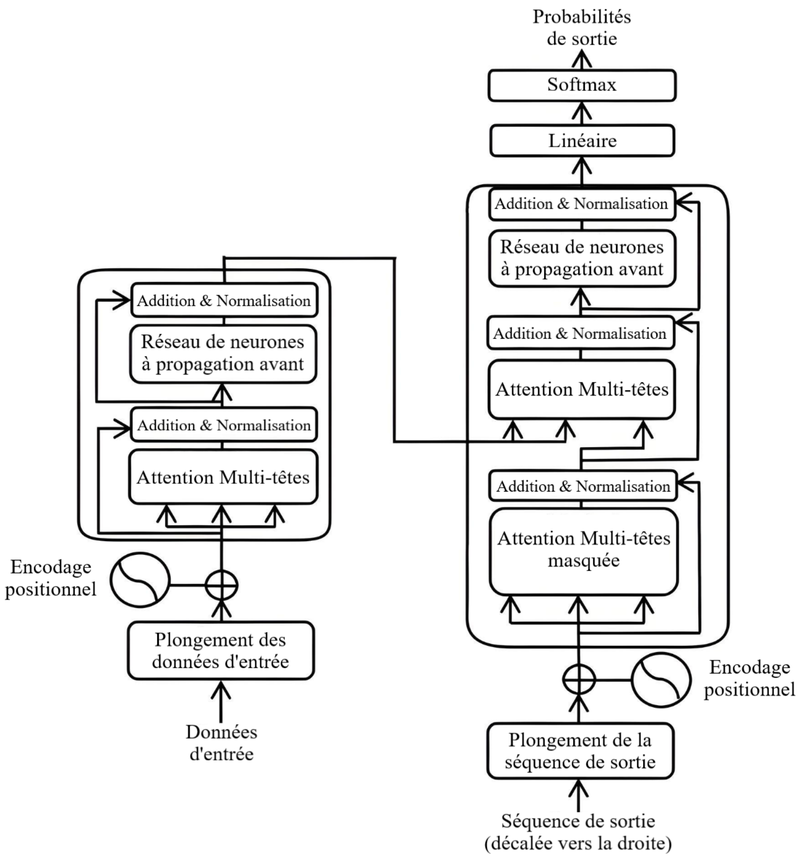
GPT4 as an encoder-decoder machine



https://next-token-visualizer.snambiar.com/

LLM

En fait, c’est bien plus compliqué!



Transformer

Entraînement

Tokenization

Tokens ≠ mots

Découpage en sous-mots

Vocabulaire limité (~50k)

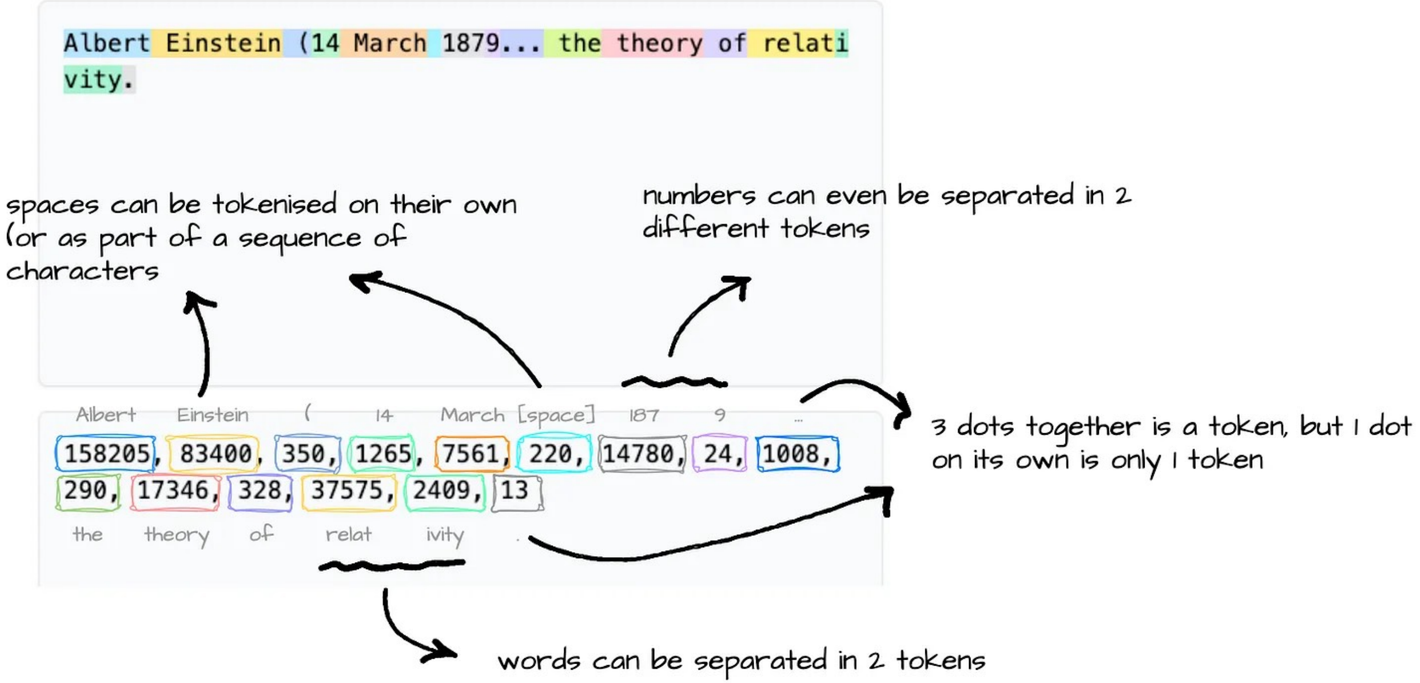
Multi-langue natif

Prédiction = token suivant

Coût = nombre de tokens

Contexte max = fenêtre en tokens

Vecteurs, pas texte brut



Entraînement

Corpus

Corpus massif : Textes divers (web, livres, articles, code, multilingue).

Qualité du corpus : Nettoyage, diversité, mais aussi biais potentiels.

Résultat : Le modèle encode des schémas de langage sans compréhension réelle.

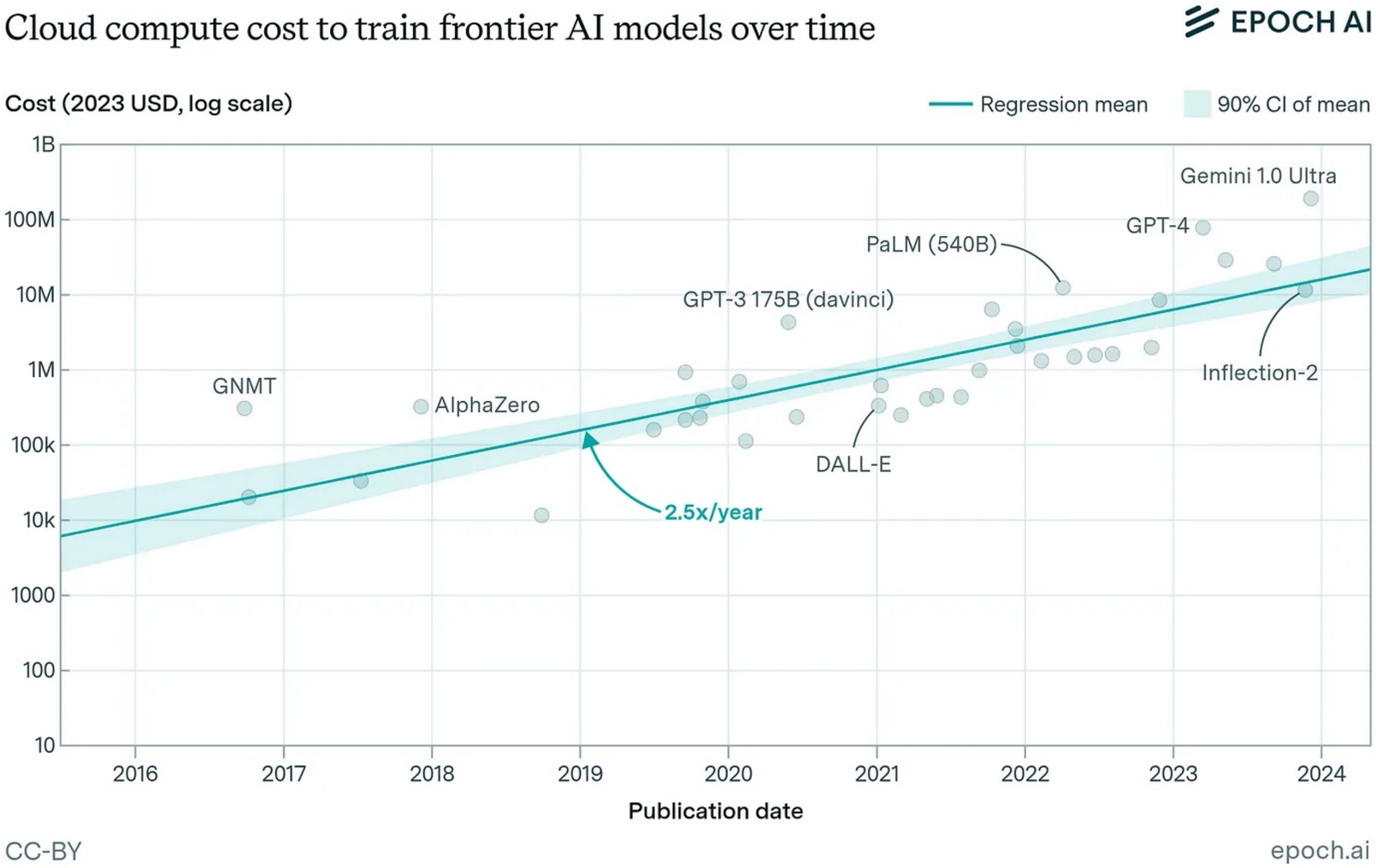
Responsabilité : Importance des sources et des impacts (biais, droits, sécurité)

Dataset	Size (GB estimated)	Weight in Training Mix	Epochs Used in 300B Training Tokens
Wikipedia	3	3%	3.40
WebText2	20	21%	2.90
Books1	12	8%	1.90
Common Crawl (filtered)	400	60%	0.44
Books2	55	8%	0.43
Total	490	100%	NA

Table: Jose Parreño • Created with Datawrapper

Entraînement

Coût



Comparaison #1

Chiffres

Modèle	Ouvert	Archi	Params (total / actifs)	#Layers	Context max	Poids approx. (bf16)	Corpus pré-training
GPT-1 (2018)	❌	Decoder Transformer	117M	12	512	~0.5 GB	BooksCorpus
GPT-2 (1.5B)	❌	Decoder Transformer	1.5B	48	1024	~6 GB	WebText (Reddit outbound links)
GPT-3 (175B)	❌	Decoder Transformer	175B	96	2048	~700 GB	Common Crawl (filtré) + WebText2 + Books + Wikipedia
GPT-4	❌	Transformer multimodal	N/D	N/D	8k–32k (selon version)	N/D	Public + licencié + données créées (non détaillé)
GPT-4o	❌	Transformer multimodal	N/D	N/D	128k	N/D	Non détaillé
GPT-5	❌	Transformer multimodal	N/D	N/D	~400k	N/D	Non publié
Claude 3 / 4	❌	Transformer multimodal	N/D	N/D	200k–1M (beta)	N/D	Internet public + accords commerciaux
Mistral Large 2 (123B)	✅	Dense decoder	123B	~80–96*	128k	~250–300 GB	Non détaillé publiquement
Mixtral 8×7B	✅	MoE decoder	47B total / 13B actifs	32 (experts 7B)	32k	~90 GB	Non détaillé publiquement
DeepSeek-V3	✅	MoE + MLA	671B / 37B actifs	61	128k	~1.3 TB	14.8T tokens (divers web + livres)
Llama 3.1 70B	✅	Dense decoder	70B	80	128k	~140 GB	~15T tokens (sources publiques)
Qwen2.5 72B	✅	Dense decoder	72.7B	80	131k	~145 GB	~18T tokens

→ Taille ≠ qualité

* ⚠️ Généré dans Chat GPT5

Comparaison #2

Benchmarks

Model	Organization	Global Average	Reasoning Average	Coding Average	Agentic Coding Average	Mathematics Average	Data Analysis Average	Language Average	IF Average
Claude 4.6 Opus Thinking High Effort	Anthropic	76.33	88.67	78.18	61.67	89.32	69.89	83.27	63.31
Claude 4.5 Opus Thinking High Effort	Anthropic	75.96	80.09	79.65	63.33	90.39	74.44	81.26	62.55
Claude 4.6 Sonnet Thinking Medium Effort	Anthropic	75.47	84.77	79.27	60.00	86.99	77.95	76.10	63.22
GPT-5.2 High	OpenAI	74.84	83.21	76.07	51.67	93.17	78.16	79.81	61.77
GPT-5.2 Codex	OpenAI	74.30	77.71	83.62	51.67	88.77	78.20	73.68	66.45
GPT-5.1 Codex Max High	OpenAI	73.98	83.65	80.68	53.33	83.22	70.12	76.48	70.38
Gemini 3 Pro Preview High	Google	73.39	77.42	74.60	55.00	81.84	74.39	84.62	65.85
Gemini 3 Flash Preview High	Google	72.40	74.55	73.90	40.00	84.17	74.77	84.56	74.86
GPT-5.1 High	OpenAI	72.04	78.79	72.49	53.33	86.90	69.61	79.26	63.90
GPT-5 Pro	OpenAI	70.48	81.69	72.11	51.67	86.17	57.04	80.69	63.96

https://livebench.ai

Contexte

Un LLM ne voit que son contexte

- Suite de tokens fournis en entrée
- Fenêtre limitée
- Aucune mémoire en dehors

Le contexte détermine la réponse

Le modèle prédit le prochain token
à partir de ce qu'il voit dans la fenêtre.

→ Qualité = qualité du contexte
→ Manque d'info = réponse fragile

Le contexte est limité

- Taille finie (en tokens)
- Fenêtre glissante
- Informations anciennes oubliées

→ Ce qui sort de la fenêtre n'existe plus

Un LLM n'a pas de mémoire.
Il a une fenêtre.

Selon la plate-forme, un bloc de texte est injecté avant la conversation et/ou avant chaque nouvelle entrée. Il contient par example des règles de comportement:

- Contexte système (i.e. API pour appeler les outils)
- Mémoire (résumé)
- Ton à adopter
- Contraintes de sécurité
- Limites
- Objectif général de la plateforme

Ce qu’un LLM ne fait pas!

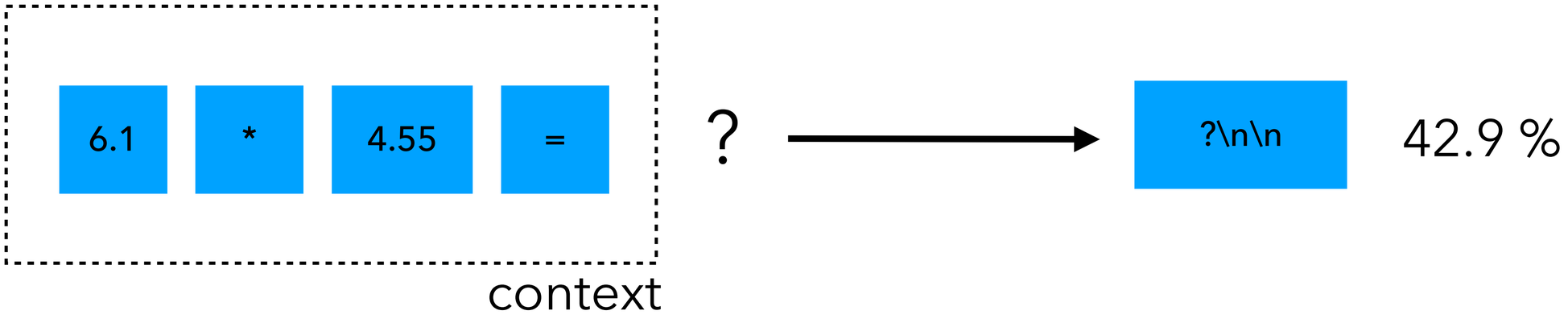
Calcul exact : Pas de moteur de calcul intégré, ce n’est pas une calculatrice.

Raisonner logiquement : Pas de raisonnement formel ou de preuve mathématique.

Connaissances en temps réel : Leur savoir s’arrête à la date de leur entraînement.

Compréhension réelle : Ils produisent des suites probables de mots, sans conscience ni compréhension profonde.

Actions autonomes : Ils ne décident rien, ne prennent pas d’initiatives ni n’agissent par eux-mêmes.



Limites

Hallucinations, vérité, raisonnement

Hallucinations

- Raisonnement plausible ❌ base fausse
- Limites à garder en tête
- code: API ou paramètre inventé

Faux positifs

- Problème inexistant
- Contexte mal interprété
- Suggestion ≠ vérité

Solution: Mitigation

- Sortie = proposition
- Tests + compilation + exécution
- Revue humaine
- Petites itérations

Contournement des limites

Outils/fonctions

Tool/Function calling: le modèle émet un *appel d'outil structuré* (arguments), pas juste du texte
Orchestration : *LLM → appel outil → résultat → LLM* (le LLM reformule et intègre)
RAG (Retrieval-Augmented Generation) : récupère des infos (docs, DB, search) pour des réponses
Exécution de code / calcul : délègue les opérations exactes (calcul, data, scripts) à un runtime
Web search / web fetch : accès *temps réel* (actualités, specs, etc.)
Vision & documents : analyse image/PDF (extraction, compréhension, QA) via outils **multimodaux**
Actions / APIs : exécute des tâches métier (tickets, CI, DB, CRM) via fonctions/outils

Appel d’outils et de functions

Exemple d’échange avec une recherche web

```
Utilisateur > Peux-tu chercher les tendances IA actuelles ?

IA > Je vais regarder ça pour toi. Un instant…

IA > {action: "web" ; parameter : "tendances IA"}

Outil > ${`action: web ; answer : "blah blah"`}

IA > Voilà, j’ai trouvé des articles sur les tendances IA de 2026. Voici quelques points
clés…

…
```

Thinking

Idée

- Le modèle génère **des étapes intermédiaires**
- Puis produit une **réponse finale**

Ce que ça améliore

- Problèmes **multi-étapes** (planification, logique)
- **Analyse** et synthèse structurée
- **Vérification** / auto-contrôle (moins d'oublis)

Effet

- Augmente la probabilité d'une réponse **cohérente**

Coût

- Plus de tokens
- Plus lent

Pourquoi ça marche bien pour le code?

Le code est prévisible

- Syntaxe stricte
- Patterns récurrents
- Projets cohérents

→ Suite probable souvent correcte

Très fort pour

- Boilerplate
- “Même style”
- Tests standards
- Refactoring local
- Explication de code

Fragile quand

- Vision système
- Cas limites
- Bugs complexes

Questions ?